



正则语言

2026

红岩



正则语言

语言 L 是正则语言，当且仅当存在 Σ 上的DFA、NFA、 ε -NFA或者RE（统称为语言识别器），接受 L 。

- 正则语言的“封闭性”问题；
 - 关于运算是否封闭
- 断言某语言不是正则语言（“**泵引理**”）；
- 正则语言的空性、成员性；
- 正则语言判定性质的复杂度；
- DFA “**最小化**” 问题。



4.1 正则语言的封闭性

- 一般来说，语言集合（语言簇）对于某个运算是封闭的，如果运算结果仍然属于这个簇。
 - 正则语言在连接、并、闭包运算下都是封闭的。
根据是什么？（从定义考虑）
 - 转换为自动机。
- 可以证明，正则语言对于以下运算都是封闭的：
 - 交、补、差、逆、同态、逆同态
- 是否存在语言簇对于某运算不是封闭的？
 - 的确是这样的话，运算结果不是正则语言。



基于DFA证明补运算封闭

- 定理3.5 如果 L 是字母表 Σ 上的正则语言, 则 $C_{\Sigma^*}L = L^C = \Sigma^* \setminus L$ 也是正则语言。
- 设DFA $A = (Q, \Sigma, v, q_0, F)$ 识别 L ,
则DFA $B = (Q, \Sigma, v, q_0, Q \setminus F)$ 识别 L^C ,
因为 $L(A) = \{w \mid \tilde{v}(q_0, w) \in F\}$, $L(B) = \{w \mid \tilde{v}(q_0, w) \in Q \setminus F\}$,
 $L(B) = C_{\Sigma^*}L(A)$ 。
- 注意用于定理证明的DFA A 是完全形DFA。
- 对于最简形DFA A 会怎样?
 - 输入串 w 导致 A 突然死亡, 因而 A 不接受 w 。那么在 B 中依然如此。其结果是 A 和 B 都不接受 w 。 (错误)
 - A 和 B 的字母表不同, 可将二者共同超集作为各自的字母表。 (比如多出一个陷阱状态)



交运算的封闭性

- 定理3.6 如果 L 和 M 都是正则语言，那么 $L \cap M$ 也是
- 间接证明：
 - 通过DeMorgan律： $L \cap M = (L^C \cup M^C)^C$ ，因为补运算和并运算是封闭的，那么它们的运算结果 $(L^C \cup M^C)^C$ 也是。
- 直接证明：
 - 通过为 $L \cap M$ 构造乘积自动机来证明。



为 $L \cap M$ 构造 DFA

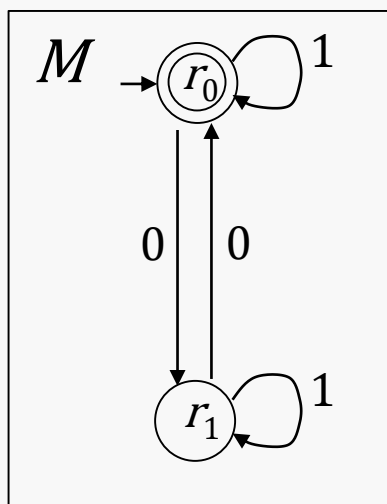
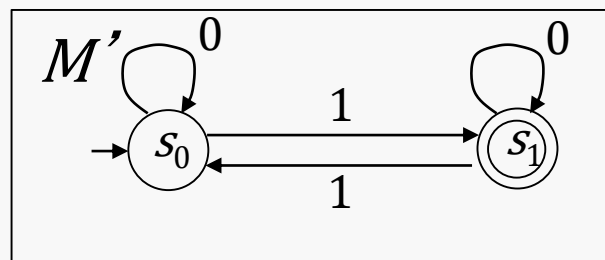
- 已知 DFA $A_L = (Q_L, \Sigma, v_L, q_L, F_L)$, $L = L(A_L)$
- 已知 DFA $A_M = (Q_M, \Sigma, v_M, q_M, F_M)$, $M = L(A_M)$
- 则 DFA $A_{L \cap M} = (Q_L \times Q_M, \Sigma, v, (q_L, q_M), F_L \times F_M)$
 - 其中状态为二元组, 即 $Q_L \times Q_M$ 的成员。
 - $v((p, q), a) = (v_L(p, a), v_M(q, a))$, 其中 $p \in Q_L, q \in Q_M$
 - 注: 这里三个 DFA 都是完全形 DFA
- 对 $w \in \Sigma^*$ 的长度归纳证明 $\tilde{v}((p, q), w) = (\tilde{v}_L(p, w), \tilde{v}_M(q, w))$
 - $A_{L \cap M}$ 接受 w 当且仅当 $\tilde{v}((q_L, q_M), w) \in F_L \times F_M$
 - 也就是 $\tilde{v}_L(q_L, w) \in F_L$, 并且 $\tilde{v}_M(q_M, w) \in F_M$
 - 即, w 同时为 A_L 和 A_M 接受



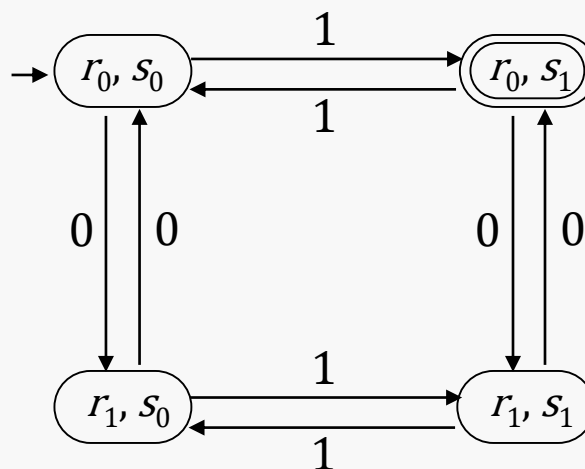
例：构造乘积DFA

$$v((p, q), a) = (v_L(p, a), v_M(q, a))$$

$L' = \text{奇数个1}$



$L = \text{偶数个0}$



$L \cap L' = \text{奇数个1 并且 偶数个0}$



3.4.1 正则语言的泵引理

- 观察有 n 个状态的DFA $A=(Q, \Sigma, v, q_0, F)$:
 - ① 若 $L(A)$ 所有成员的长度都小于 n , 那么 $L(A)$ 是正则语言。
 - ② 若 $w \in L(A)$, $|w| \geq n$, 那么 A 中有 $\text{PATH}[q_0, q, w]$, $q \in F$,
 - $\text{PATH}[q_0, q, w]$ 有 $|w|+1$ 个顶点, 并且其中至少两个顶点必然相同。(鸽巢原理)
 - 如果 p 就是这样的顶点, 那么 $\text{PATH}[q_0, q, w]$ 可分段为 $\text{PATH}[q_0, p, x]$, $\text{PATH}[p, p, y]$, $\text{PATH}[p, q, z]$ 。
 - 那么, 进一步地, 必然有 $xy^kz \in L(A)$, $k \geq 0$ 。
- 语言 L 不是正则语言, 当且仅当没有能识别 L 的DFA存在 (隐含着NFA和RE都不存在)。

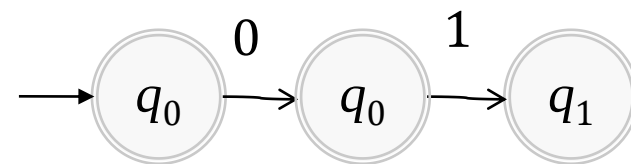
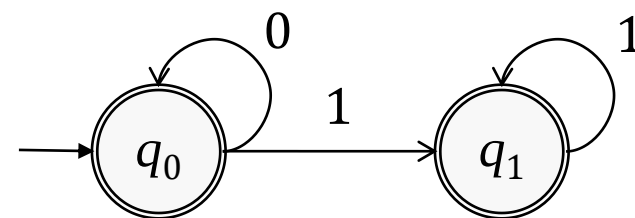
注意, 在DFA中, $\text{PATH}[q_0, q, w]$ 即 $\tilde{v}(q_0, w)=q$;
 $\text{PATH}[q_0, p, x]$ 即 $\tilde{v}(q_0, x)=p$; $\text{PATH}[p, p, y]$ 即 $\tilde{v}(p, y)=p$;
 $\text{PATH}[p, q, z]$ 即 $\tilde{v}(p, z)=q$ 。



讨论 $\text{PATH}[q_0, q, w]$ 分三段的情形

- $w=xyz$, $|w| \geq n$, $n=|Q|$:
 - $\text{PATH}[q_0, p, x]$, $|x| \geq 0$, $|xy| \leq n$ (因为 p 也可作为 q_0)
 - $\text{PATH}[p, p, y]$, $|y| \geq 1$, $|xy| \leq n$ (因为至少有两个 p 顶点)
 - $\text{PATH}[p, q, z]$, $|xyz| \geq n$
 - 满足 $w=xyz$, $xy^kz \in L(A)$, $k \geq 0$ 。

- 例: $L = \{0^m 1^l \mid m, l \geq 0\}$
 - 对于 $n=2$, $w=01 \in L$, $w=xyz$, 其中,
 - $x=\varepsilon$, $y=0$, $z=1$
 - $|xy| = |0| \leq n$, $|w| = |xyz| \geq n$
 - $xy^kz \in L$, $k \geq 0$, 即 $\varepsilon 0^k 1 \in L$, $k \geq 0$



- 一般地, $w = 0^m 1^l$, 其中 $m+l \geq 2$, $m, l \geq 0$
 - w 有: 000^*1^* 、 011^* 、 111^* , 故 x 有?



正则语言的泵引理

- 定理3.3 (泵引理) 设 L 是正则语言, 则存在正整数 n , 对任意 $w \in L$, 如果 $|w| \geq n$, 那么有分段 $w = xyz$, 使得:
 - ① $y \neq \varepsilon$
 - ② $|xy| \leq n$
 - ③ $xy^kz \in L, k \geq 0$
- 正则语言满足泵引理; 满足泵引理结论的不一定是正则语言。
- 不满足泵引理必定不是正则语言。
- 泵引理常用于证明语言 L 是非正则语言 (反证法):
 - 对于任意正整数 n , 存在 $w \in L$, $|w| \geq n$, 对满足条件①和②的任意 $w = xyz$, 不满足条件③。



例：证明 $\{0^m 1^m | m > 1\}$ 不是正则语言

- $L = \{0^m 1^m | m > 1\}$
- 对于任意正整数 n ，选 $w = 0^n 1^n$ ， $w \in L$ ， $|w| \geq n$ ，
- 设 $w = xyz$ ，其中 $x = 0^{m-k}$ ， $y = 0^k$ ， $z = 0^{n-m} 1^n$ ， $m \leq n$ ， $k > 0$ ，则有：
- ① $y \neq \varepsilon$
- ② $|xy| \leq n$
- ③ 对于 $i=0$ ， $xy^i z = 0^{n-k} 1^n \notin L$ ，矛盾



证明泵引理

- 证明：假设 L 是正则语言，那么存在DFA $A=(Q, \Sigma, v, q_0, F)$ 有 $L=L(A)$ 。
- 不妨假设 A 有 n 个状态， $|Q|=n$ 并考虑长度不小于 n 的串 $w=a_1\cdots a_m$ ，其中 $m \geq n$ 并且 $a_1, \cdots, a_m \in \Sigma$ 。
- 对于 $i=0, 1, \cdots, n$ 定义状态 $p_i=\tilde{v}(q_0, a_1\cdots a_i)$ ，注意， $p_0=q_0$ 。
- 由鸽巢原理， p_0 到 p_n 这 $n+1$ 个状态不可能都不同，因为 Q 只有 n 个元素。因此，我们能够找到两个不同的非负整数 i 和 j ，满足 $0 \leq i < j \leq n$ ，使得 $p_i=p_j$ 。
- 至此，取 $w=xyz$ ，其中 $x=a_1\cdots a_i$ ， $y=a_{i+1}\cdots a_j$ ， $z=a_{j+1}\cdots a_m$ ，得到 $xy^kz \in L$ ， $k \geq 0$ 。



例4.3

- 证明 $L = \{1^p \mid p \text{ 是素数}\}$ 不是正则语言。
- 反证法，假定 L 是正则的，
- 考虑某个素数 $q \geq n+2$ ，其中 n 是泵长度
- 选择串 $w = 1^q$ ，我们可以写成 $w = xyz$ 使得 $y \neq \varepsilon$ 且 $|xy| \leq n$
- 令 $|y| = m$ ，那么 $|xz| = q - m$ ，考虑串 $s = xy^{q-m}z$ 的长度
- $|xy^{q-m}z| = |xz| + (q-m)|y| = q - m + (q-m)m = (m+1)(q-m)$
- 注意到 $m+1 > 1$ ，因为 $y \neq \varepsilon$
- 另注意到 $q \geq n+2$ 则 $q-m > 1$ ，（ $m \leq n$ ）
- 因此， s 的长度不是素数，矛盾。



3.6 正则语言的判定性质

- 对于 L 找到语言识别器接受 L ，当且仅当 L 是正则语言。
- 语言识别器：DFA、NFA、 ε -NFA、RE
- 判定语言空性
- 判定语言成员性
- 判定语言等价性
- 判定语言有限性



空性测试

- 为语言构建识别器，并运行空性测试算法，即得结果。

输入：识别 L 的DFA (Q, Σ, v, q_0, F)

输出：真（空）或假

$S = \{q_0\};$

while (S 中存在一个未标记元素 q) {

 标记 q ;

$T = \{v(q, a) \mid a \in \Sigma\}$

$S = S \cup (T - S)$

return $S \cap F = \emptyset$

// S 为可达状态集合



基于NFA测试语言L是否为空的算法

输入：识别 L 的NFA (Q, Σ, v, q_0, F)

输出：真（空）或假

$S = \omega(\{q_0\});$

while (S 中存在一个未标记元素 q) {

 标记 q ;

$T = \omega(\cup a \in \Sigma \cdot v(q, a));$

$S = S \cup (T \setminus S)$

return $S \cap F = \emptyset$

// S 为NFA的所有可达状态的集合



基于正则表达式判定语言空性

- 基础：运算符个数为 0 的正则表达式匹配语言， $L(\emptyset)$ 为空， ε 和 a 的语言不空。
- 归纳：假设运算符个数小于 k 的正则表达式 r 的空性已知，那么运算符个数为 k 时，正则表达式为以下四种情形：
 - $r=r_1+r_2$ ， $L(r)$ 为空当且仅当 $L(r_1)$ 和 $L(r_2)$ 都为空；
 - $r=r_1r_2$ ， $L(r)$ 为空当且仅当 $L(r_1)$ 或 $L(r_2)$ 为空；
 - $r=r_1^*$ ， $L(r)$ 不空；
 - $r=(r_1)$ ， $L(r)$ 为空当且仅当 $L(r_1)$ 为空。



判定语言的成员性

- DFA、NFA的判定性质已有算法表示。
- 时间复杂度：
 - DFA $O(|w|)$
 - NFA $O(|w||Q|^2)$ 注：考虑当前状态集合
 - ε -NFA 同上 注：计算集合的 ε 闭包
- RE
 - 转化为 ε -NFA



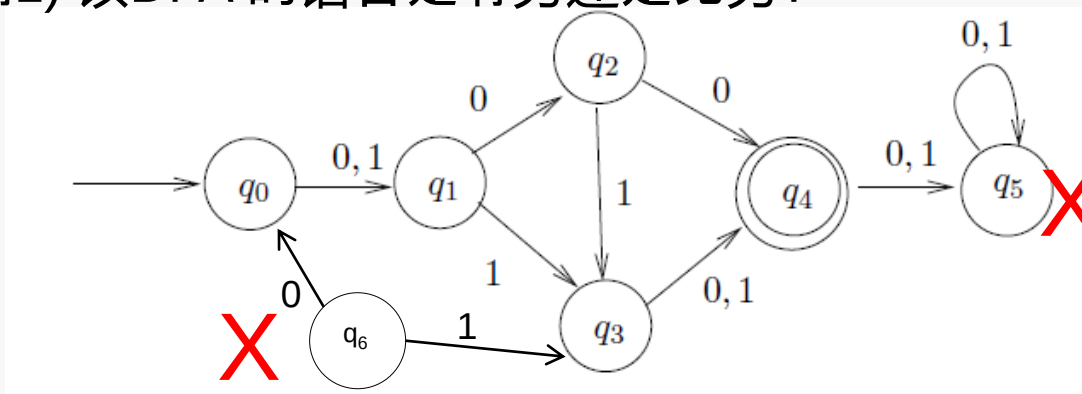
判定语言的有限性

- 输入：正则语言 L
- 输出：真（有限）或假
 1. 为 L 构建 DFA.
 2. 移除所有从开始状态不可达到的状态.
 3. 移除所有不可能到达任何接受状态的状态.
 4. 所有这些移除之后, 检查产生的 DFA 中是否有环存在
 5. 若无环则 L 有穷; 否则 L 为无穷
- 另一种方法
 - 构建正则表达式检查是否有 Kleene 闭包存在



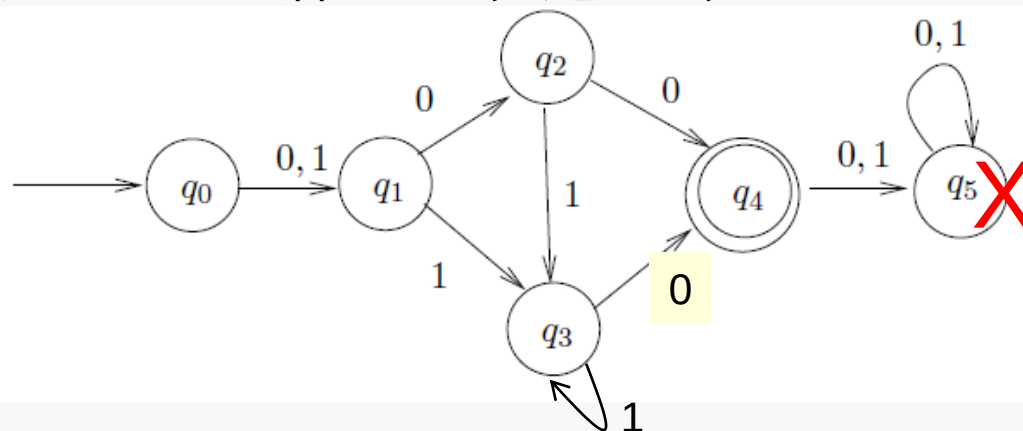
有限性测试举例

例1) 该DFA 的语言是有穷还是无穷?



有穷

例2) 这个DFA 的语言是有穷还是无穷?

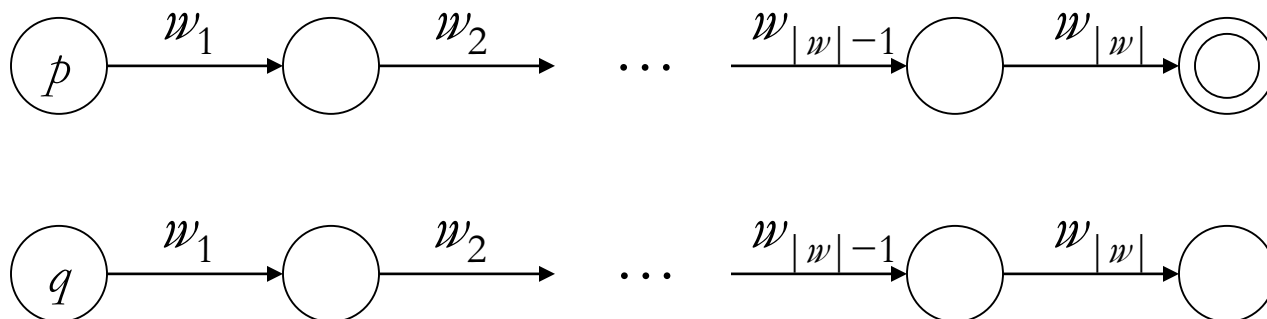


无穷



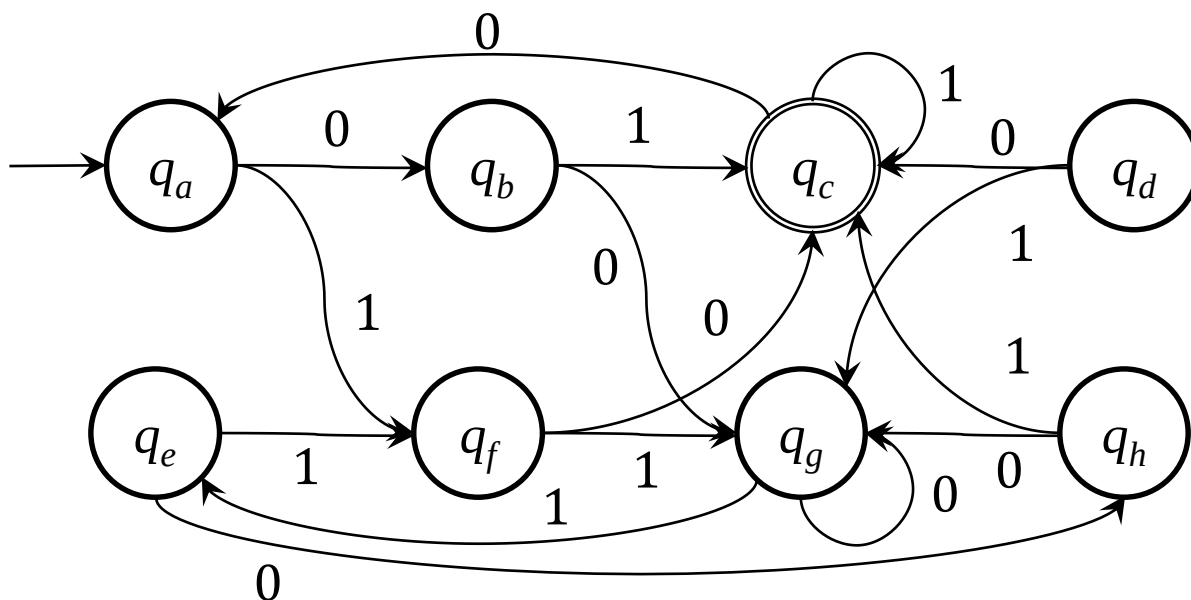
3.7 DFA最小化

- DFA (Q, Σ, v, q_0, F) 的任意两个状态之间会有什么关系？
- 状态 p 和 q 是可区分的，如果，
 - $\exists w \in \Sigma^* \cdot (\tilde{v}(p, w) \in F \wedge \tilde{v}(q, w) \notin F)$ ，或者，
 - $\exists w \in \Sigma^* \cdot (\tilde{v}(p, w) \notin F \wedge \tilde{v}(q, w) \in F)$
- 状态 p 和 q 是等价的，如果，
 - $\forall w \in \Sigma^* \cdot (\tilde{v}(p, w) \in F \leftrightarrow \tilde{v}(q, w) \in F)$





例：状态与状态等价或可区分



$(q_c, q_a)\varepsilon$	$(q_c, q_b)\varepsilon$	$(q_c, q_d)\varepsilon$	$(q_c, q_e)\varepsilon$	可区分的
$(q_c, q_f)\varepsilon$	$(q_c, q_g)\varepsilon$	$(q_c, q_h)\varepsilon$		
$(q_a, q_b)1$	$(q_a, q_d)0$	$(q_a, q_f)0$	$(q_a, q_g)01$	$(q_a, q_h)1$
$(q_b, q_d)1$	$(q_b, q_e)1$	$(q_b, q_f)1$	$(q_b, q_g)1$	
$(q_d, q_e)0$	$(q_d, q_g)0$	$(q_d, q_h)0$		<状态对><测试串>

$(q_b, q_h): 1q_c \text{ 或 } 0q_g$	$(q_d, q_f): 0q_c \text{ 或 } 1q_g$	等价的
$(q_a, q_e): 1q_f \text{ 或 } 01q_c \text{ 或 } 00q_g$		



算法3.1 发现等价对的填表算法

输入：DFA $A=(Q, \Sigma, v, q_0, F)$

输出： $\{(q, p) \mid q, p \in Q \text{ 且 } q \text{ 与 } p \text{ 是等价的}\}$

T 初始化为 $Q \times Q$ 的下三角；

foreach $q \in F$ do 标记 T 的 q 行和 q 列诸元素；

置 c 为YES；

while c do {

 置 c 为NO；

 foreach T 的未标记元素 $T[q, p]$ do

 foreach $a \in \Sigma$ do

 if ($T[v(q, a), v(p, a)]$ 有标记) {

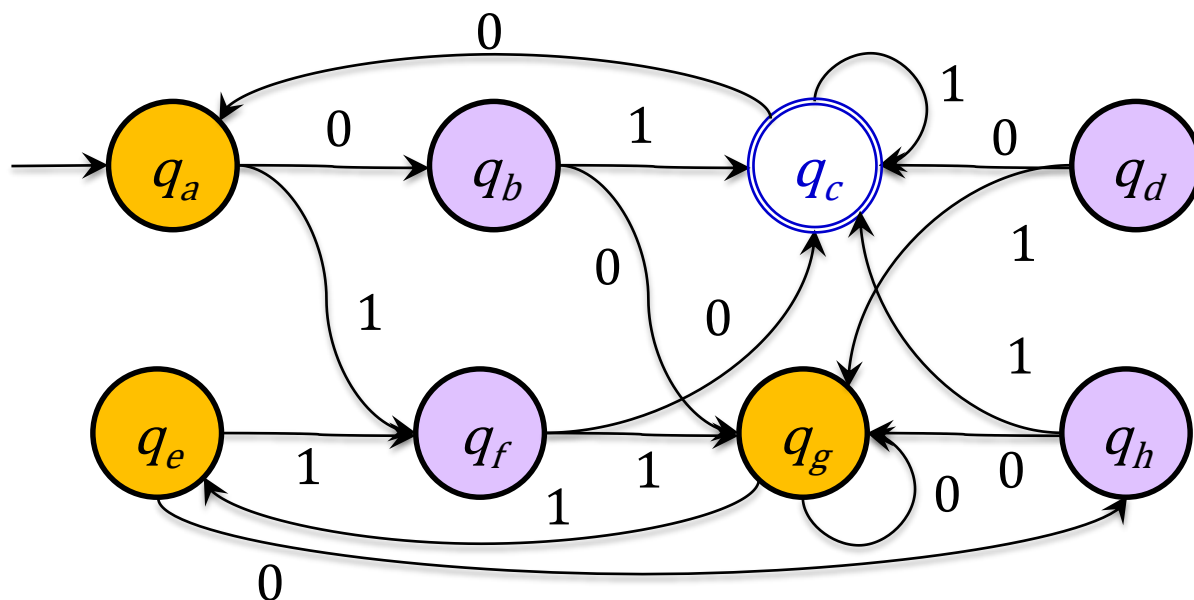
 标记 $T[q, p]$ ；

 置 c 为YES}}

return T 的所有未带标记元素。



例：填表算法

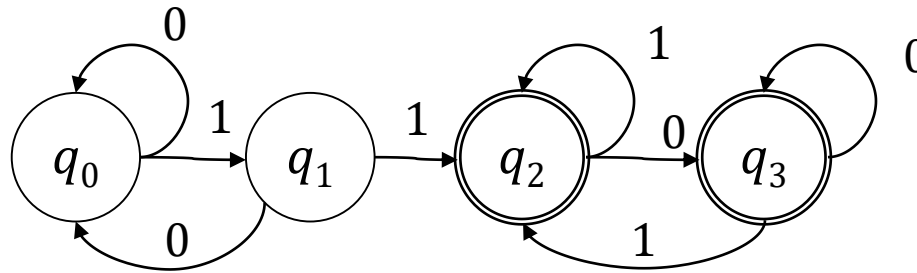


q_b	$\times^1$						
q_c	$\times$	$\times$					
q_d	$\times^0$	$\times^1$	$\times$				
q_e		$\times^1$	$\times$	$\times^0$			
q_f	$\times^0$	$\times^0$	$\times$		$\times^0$		
q_g	$\times^1$	$\times^1$	$\times$	$\times^0$	$\times^1$	$\times^0$	
q_h	$\times^1$		$\times$	$\times^0$	$\times^1$	$\times^1$	$\times^1$
	q_a	q_b	q_c	q_d	q_e	q_f	q_g

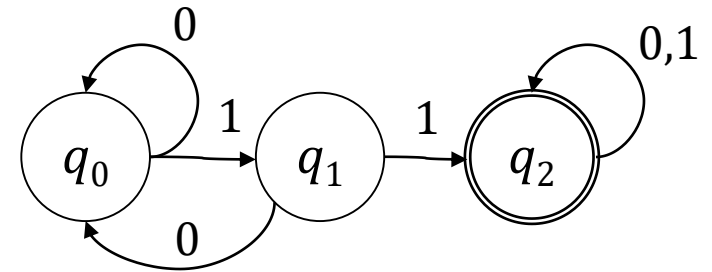
c行c列打标记（蓝色）；
对于剩余对，同标记a射出到达的对儿有标记则打标记（黑色带a上标）；
逐遍扫描直到不变为止。



例：填表算法



图(b)

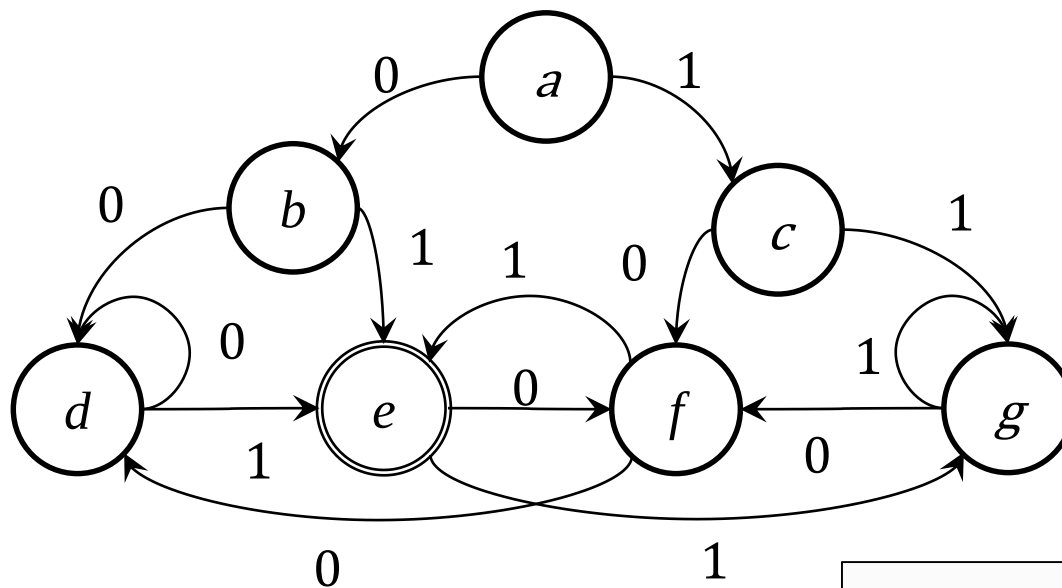


图(c)

q_1	$\times^1$		
q_2	$\times$	$\times$	
q_3	$\times$	$\times$	
	q_0	q_1	q_2

	0	1
$\rightarrow\{0\}$	$\{0\}$	$\{1\}$
$\{1\}$	$\{0\}$	$\{2,3\}$
$*\{2,3\}$	$\{2,3\}$	$\{2,3\}$

例：填表算法

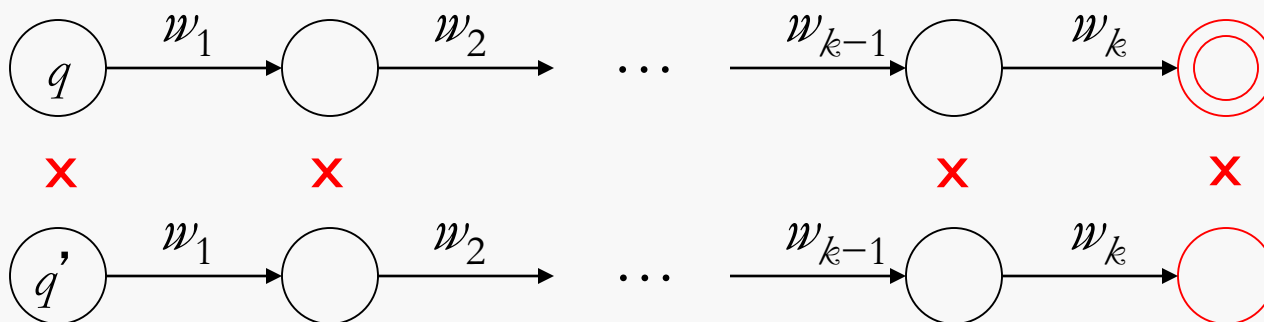


		0	1
b	$\times^1$		
c	?	$\times^1$	
d	$\times^1$		$\times^1$
e	$\times$	$\times$	$\times$
f	$\times^1$		$\times^1$
g	?	$\times^1$	?
	a	b	c

	0	1
$>\{a,g,c\}$	$\{d,b,f\}$	$\{a,g,c\}$
$\{d,b,f\}$	$\{d,b,f\}$	$\{e\}$
$*\{e\}$	$\{d,b,f\}$	$\{a,g,c\}$



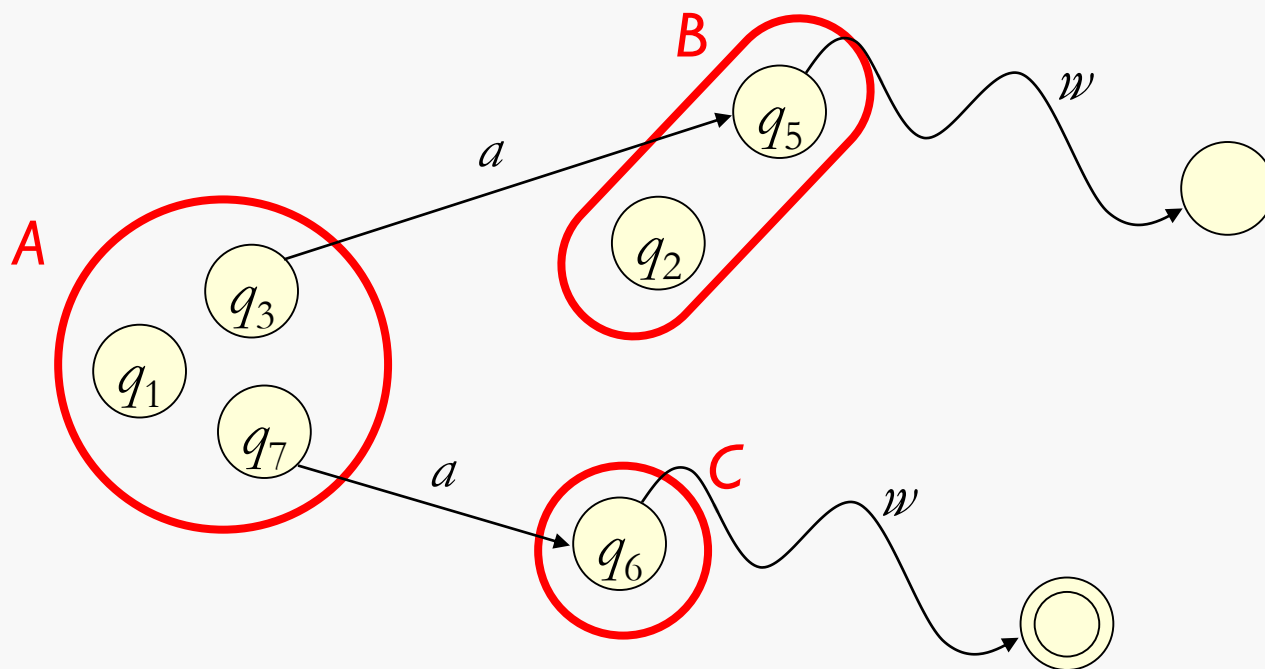
- 定理3.8 如果通过填表算法不能区分两个状态，那么它们是等价的。



反证法：假定不能发现 q 和 q'



- 为什么当我们合并等价状态时不会产生不一致性？



不一致：合并了 q_3 和 q_7 到 **A** 中

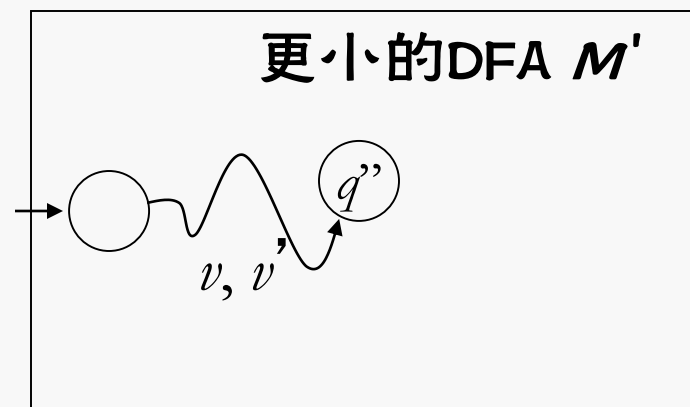
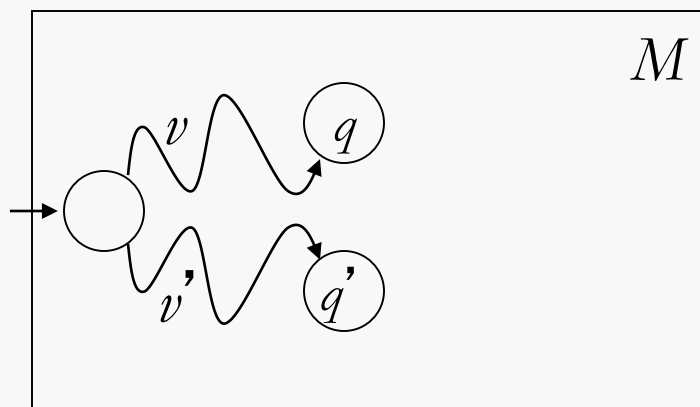
B 和 **C** 可区分，所以存在 w 一个到达接受状态另一个不能
结果 **A** 中状态不全是等价的



- 为什么没有更小的DFA了？

假定有

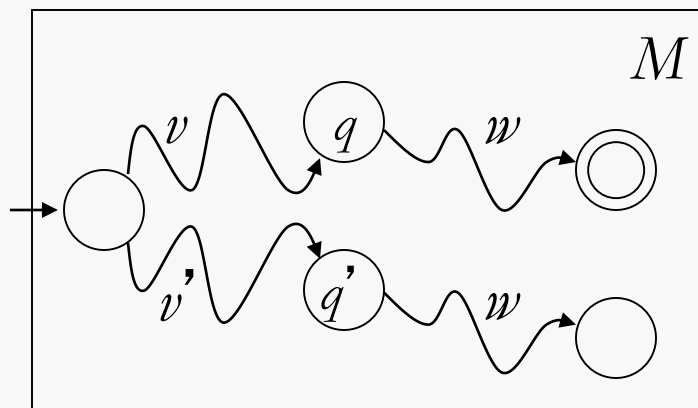
根据鸽巢原理必定出现下面情况：



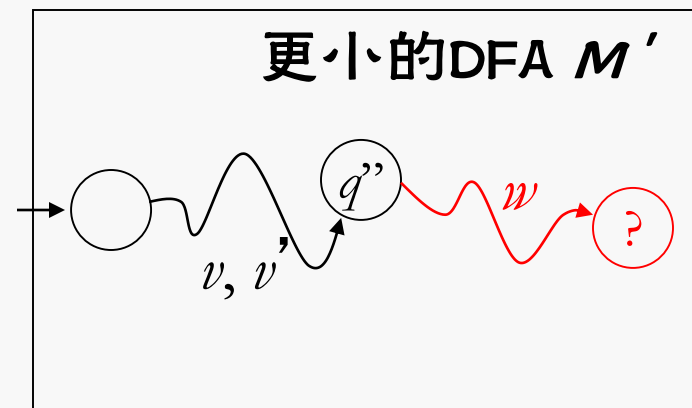


➤ 为什么没有更小的DFA了?

但是之后



每个状态序对都是可区分的。



q'' 不能存在!



算法3.2: DFA最小化的划分算法

输入: 完全形DFA (Q, Σ, v, q_0, F)

输出: DFA $(\mathcal{P}, \Sigma, \text{move}[G, a], G_0, F')$, $q_0 \in G_0$, $\forall G \in F' \cdot G \cap F \neq \emptyset$ 。

$\mathcal{P} = \{F, Q \setminus F\}$;

while($G \in \mathcal{P}$ 且 G 未标记){

$R = \{(p, q) \mid p, q \in G, \forall a \in \Sigma \cdot \exists G' \in \mathcal{P} \cdot v(p, a) \in G' \wedge v(q, a) \in G'\}$

$\mathcal{G} = G/R$ //按**最大化一致性原则**得出 G 的划分 $\mathcal{G}$

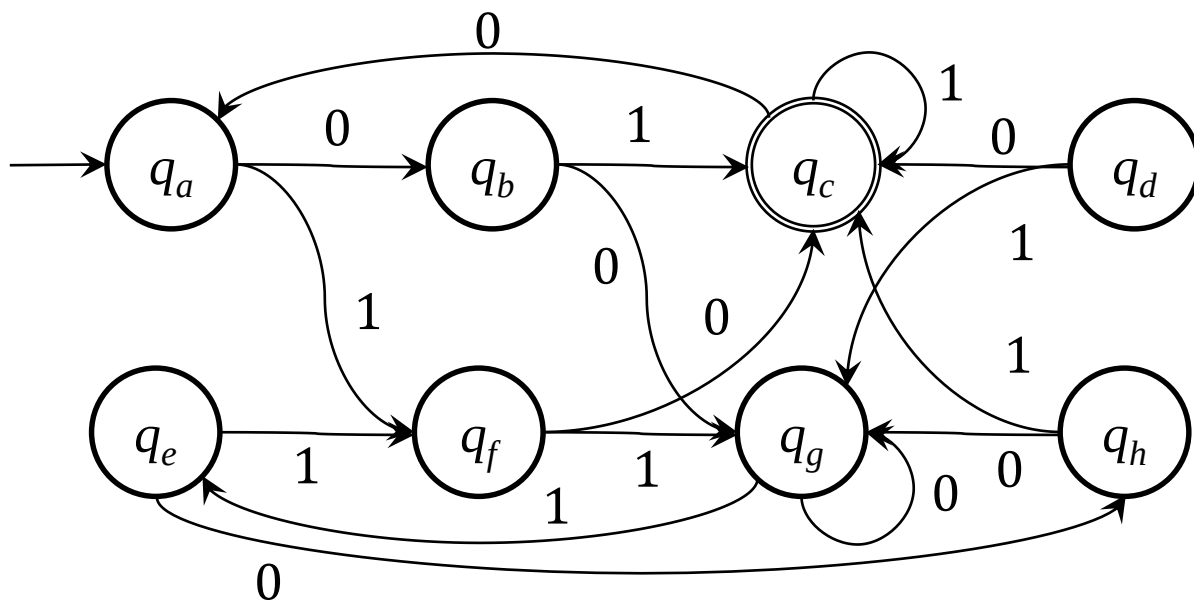
if($G \in \mathcal{G}$) 标记 G // **G 是一致的**

else $\{\mathcal{P} = \mathcal{P} \setminus G; \mathcal{P} = \mathcal{P} \cup \mathcal{G};$

清除 $\mathcal{P}$ 中每个成员的标记

}}

例：采用Hopcroft算法最小化



$G_1 = \{c\}$

$G_2 = \{a, b, d, e, f, g, h\}$

G_2	0	1
a	G_2	G_2
b	G_2	G_1
d	G_1	G_2
e	G_2	G_2
f	G_1	G_2
g	G_2	G_2
h	G_2	G_1

$G_1 = \{c\}$

$G_2 = \{a, e\}$

$G_3 = \{d, f\}$

$G_4 = \{b, h\}$

$G_5 = \{g\}$

G_2	0	1
a	G_4	G_3
e	G_4	G_3
g	G_2	G_2

$G_1 = \{c\}$

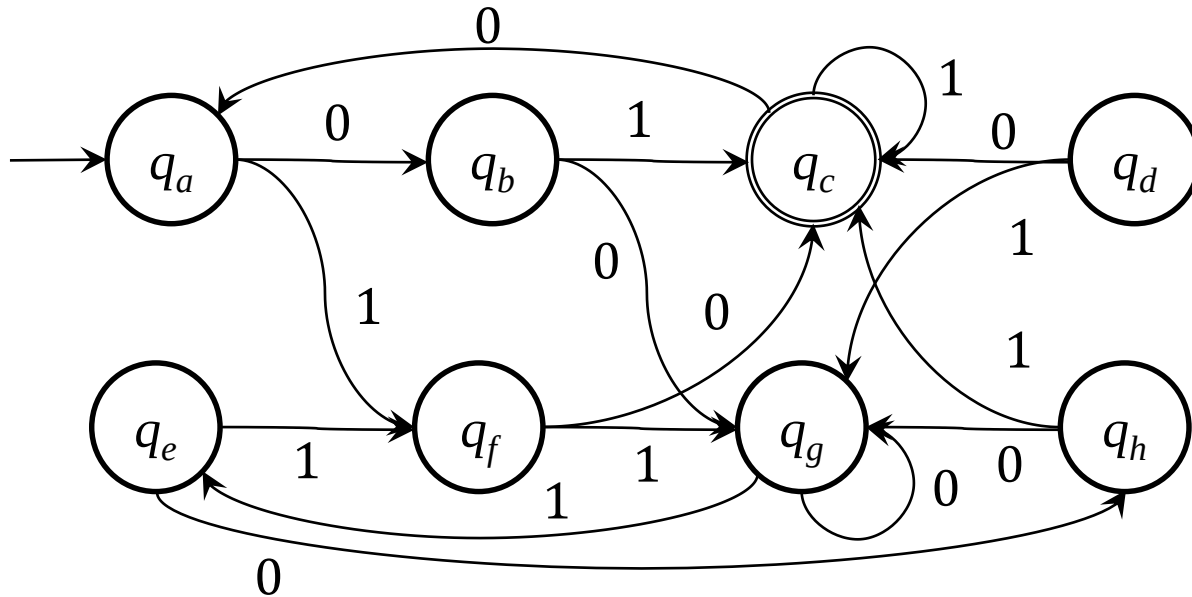
$G_2 = \{a, e, g\}$

$G_3 = \{d, f\}$

$G_4 = \{b, h\}$

检查一致性并构建转移表

2026/3/23



$G_1 = \{c\}$
 $G_2 = \{a, e\}$
 $G_3 = \{d, f\}$
 $G_4 = \{b, h\}$
 $G_5 = \{g\}$

G_1	0	1
c	G_2	G_1

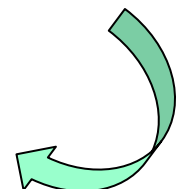
G_2	0	1
a	G_4	G_3
e	G_4	G_3

G_5	0	1
g	G_5	G_2

G_3	0	1
d	G_1	G_5
f	G_1	G_5

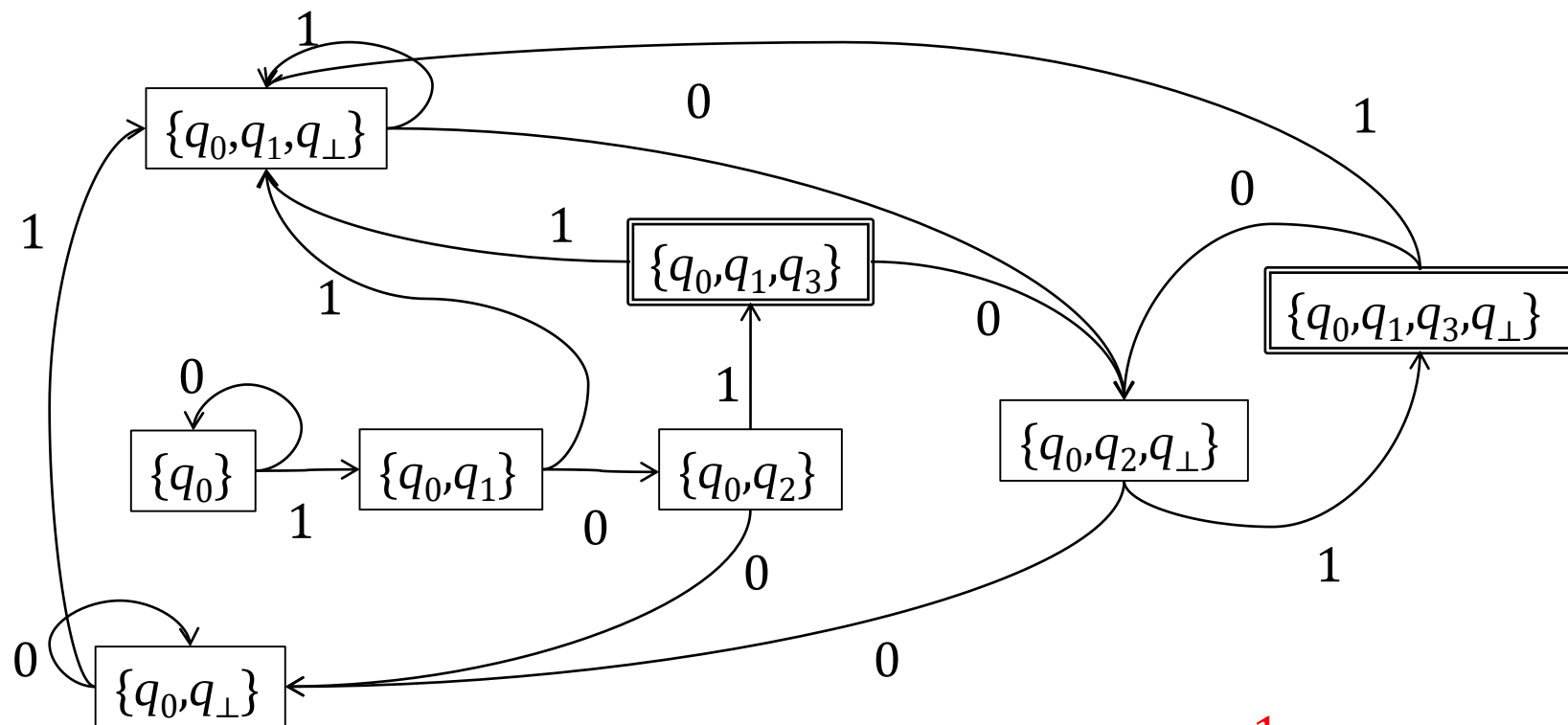
G_4	0	1
b	G_5	G_1
h	G_5	G_1

	0	1
* $G_1 = \{c\}$	G_2	G_1
$\rightarrow G_2 = \{a, e\}$	G_4	G_3
$G_3 = \{d, f\}$	G_1	G_5
$G_4 = \{b, h\}$	G_5	G_1
$G_5 = \{g\}$	G_5	G_2



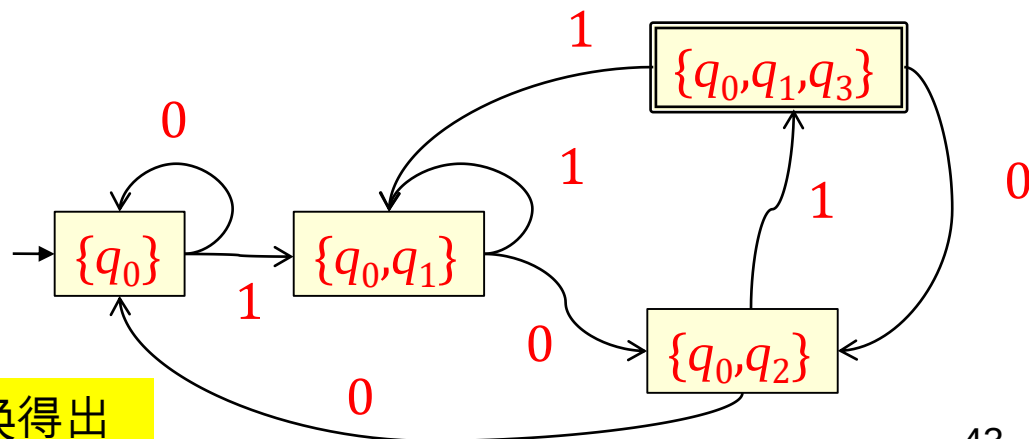


例：最小化101结尾的0-1串DFA



从完全形NFA转换得出

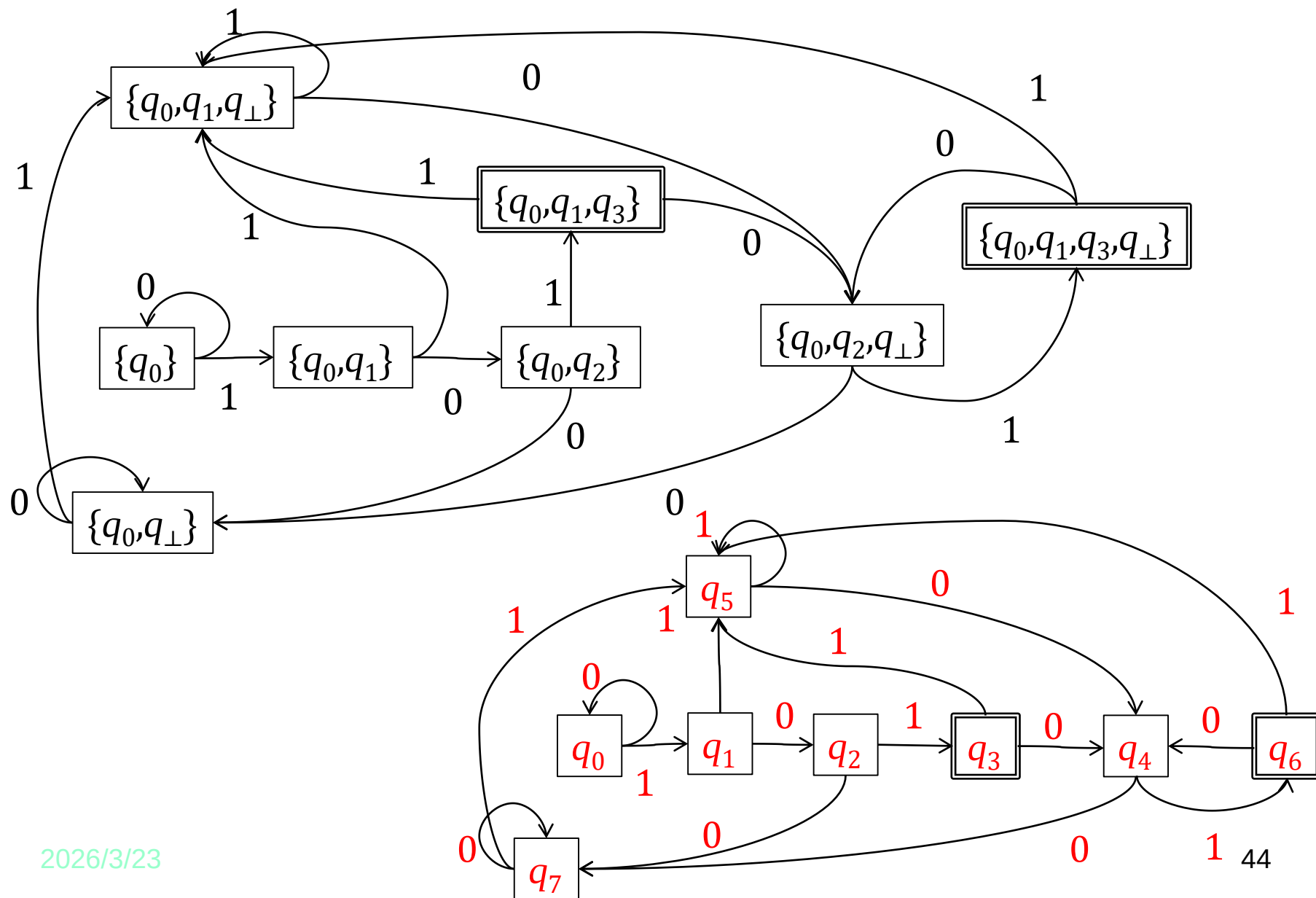
最小化
?



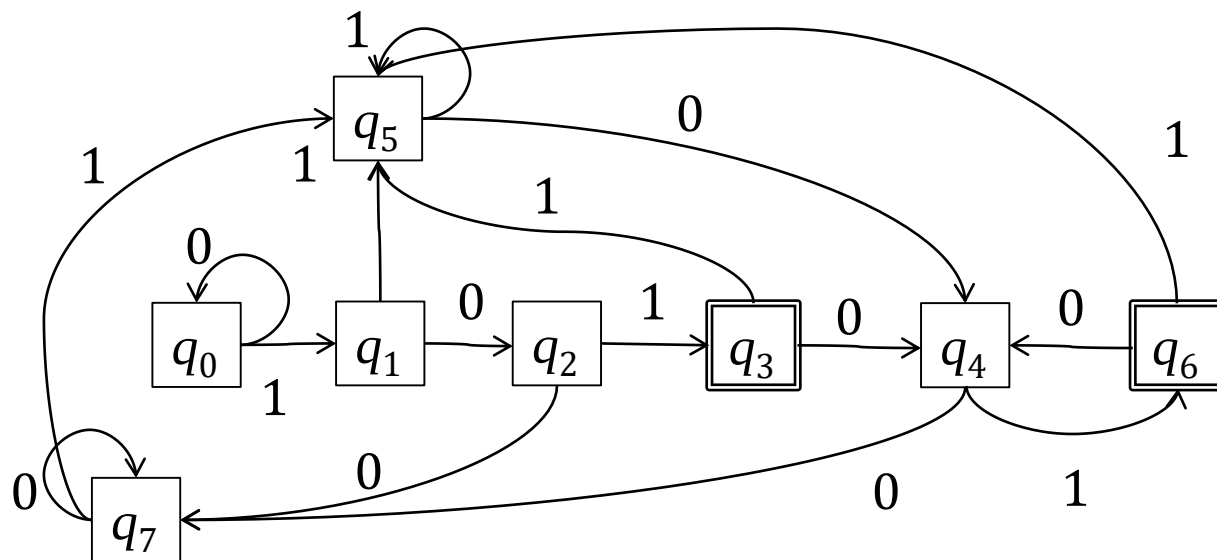
从最简形NFA转换得出



对状态命名



用划分法进行最小化



$$G_1 = \{q_3, q_6\}$$

$$G_2 = \{q_0, q_1, q_2, q_4, q_5, q_7\}$$

G_2	0	1
q_0	G_2	G_2
q_1	G_2	G_2
q_2	G_2	G_1
q_4	G_2	G_1
q_5	G_2	G_2
q_7	G_2	G_2

$$G_1 = \{q_3, q_6\}$$

$$G_2 = \{q_0, q_1, q_5, q_7\}$$

$$G_3 = \{q_2, q_4\}$$

G_2	0	1
q_0	G_2	G_2
q_1	G_3	G_2
q_5	G_3	G_2
q_7	G_2	G_2

$$G_1 = \{q_3, q_6\}$$

$$G_2 = \{q_0, q_7\}$$

$$G_3 = \{q_2, q_4\}$$

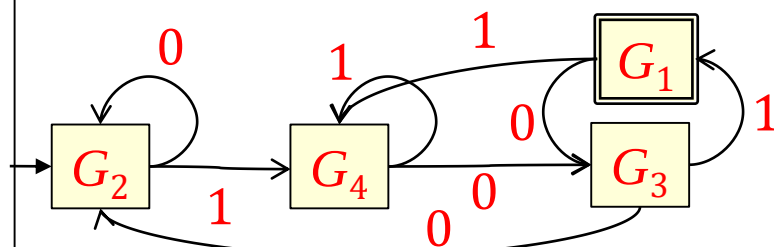
$$G_4 = \{q_1, q_5\}$$

G_2	0	1
q_0	G_2	G_4
q_7	G_2	G_4

G_3	0	1
q_2	G_2	G_1
q_4	G_2	G_1

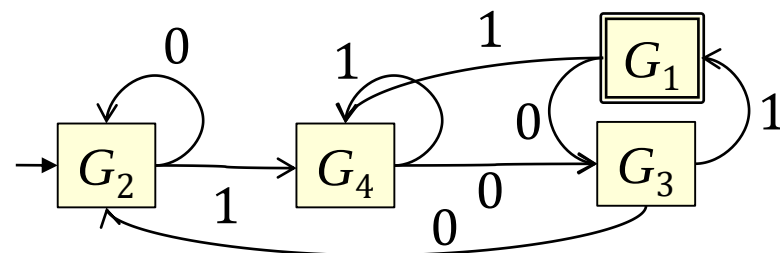
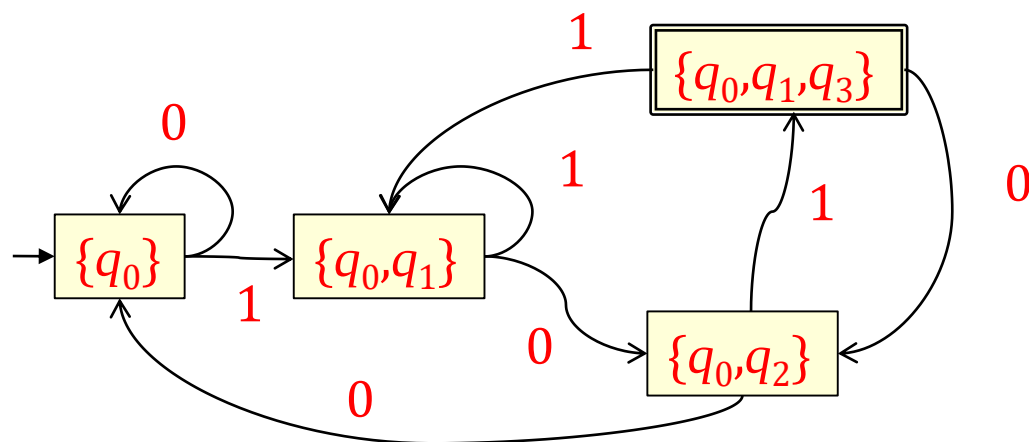
G_4	0	1
q_1	G_3	G_4
q_5	G_3	G_4

G_1	0	1
q_3	G_3	G_4
q_6	G_3	G_4





“101结尾的0-1串”最小DFA





两种方法最小化 DFA方法的比较

填表算法

- ① 找出等价状态；
- ② 合并等价状态。
- ③ 构建等价块间转移关系。
- ④ 适用于完全形DFA的最小化
- ⑤ 逐步逼近

Hopcroft算法

- ① 分裂划分块为最大化一致性划分块；
- ② 构建划分块间转移关系。
- ③ 适用于完全形DFA的最小化
- ④ 迭代



小结

- 知识点：泵引理、最小化
 - 知识点：可区分的状态、等价的状态
 - 最小化算法
-
- 习题 3.7(3)、3.10、3.11



客观题作业1

2026/3/23

